

Tài liệu thuật toán

Tài liệu riêng giải thích A* với công thức $f(n) = g(n) + h(n)$ và heuristic theo tọa độ.

A* Search

Dự án	RouteLab Group 1 - Shortest Path Visualizer
Mục tiêu	Giải thích thuật toán, contract dữ liệu, pseudocode, trace demo và checklist kiểm thử.
Dataset minh họa	Ho Chi Minh City graph, 7 nodes, source = 1, target = 6

1. Vì Sao Cần A*?

Dijkstra không biết target nằm ở đâu nên có thể mở rộng nhiều node không cần thiết. A* thêm heuristic $h(n)$ để ưu tiên node có vẻ gần target hơn, từ đó mô phỏng cách tìm đường trên bản đồ trực quan hơn.

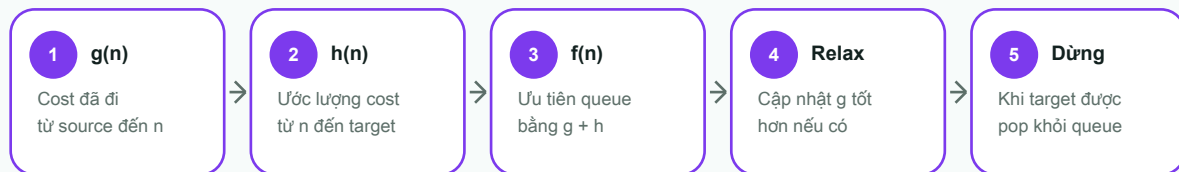
A* priority

$$f(n) = g(n) + h(n)$$

$g(n)$: chi phí từ source đến n

$h(n)$: ước lượng từ n đến target bằng tọa độ

Nếu heuristic không vượt quá chi phí thật còn lại, A* vẫn tìm được shortest path.



Priority queue minh họa



Node có độ ưu tiên nhỏ nhất được lấy ra trước; sau đó thuật toán relax các cạnh đi ra.

2. Heuristic Trong Project

Demo dùng tọa độ lat/long của node để tính khoảng cách đường chim bay đến target, sau đó scale xuống để giữ ước lượng bảo thủ trên graph giả lập. Khi heuristic không vượt chi phí thật còn lại, A* vẫn tìm shortest path tối ưu.

Node	Tên	$h(\text{node}, \text{target})$
0	Central Post Office	0.579
1	Ben Thanh Market	0.979
2	Independence Palace	0.857
3	Turtle Lake	0.655
4	Tan Dinh Church	0.907
5	War Remnants Museum	0.933
6	Saigon Zoo	0.000

```

function heuristic(node, target):
    return haversineDistance(node.latLng, target.latLng) * conservativeScale

priority(node) = gScore[node] + heuristic(node, target)
  
```

3. Pseudocode Và Cách Cài Đặt

```

gScore = map with infinity for every node
previous = empty map
gScore[source] = 0
queue.push(source, priority = heuristic(source, target))

while queue is not empty:
    current = queue.popMin()
    if current == target:
        break

    for each edge current -> neighbor:
        tentativeG = gScore[current] + edge.weight
        if tentativeG < gScore[neighbor]:
            previous[neighbor] = current
            gScore[neighbor] = tentativeG
            fScore = tentativeG + heuristic(neighbor, target)
            queue.push(neighbor, priority = fScore)

return reconstructPath(previous, source, target)

```

Các điểm cài đặt cần chú ý:

- A* không thay thế validator: edge weight vẫn phải không âm.
- Nếu heuristic quá cao, A* có thể mất bảo đảm tối ưu.
- Khi $h(n) = 0$ cho mọi node, A* trở thành Dijkstra.
- Frontend nên hiển thị g, h, f để người xem hiểu vì sao một node được ưu tiên.

4. Trace Trên Dataset HCM 7 Nodes

Bước	Sự kiện	Cạnh	Cost tích lũy
1	Relax edge	1 → 2	2.5
2	Relax edge	1 → 0	3.4
3	Relax edge	1 → 5	4.6
4	Relax edge	2 → 5	3.8
5	Relax edge	2 → 3	4.7
6	Relax edge	5 → 4	7.8
7	Relax edge	3 → 4	7.4
8	Relax edge	3 → 6	7.5
Done	Visited: 1 → 2 → 0 → 5 → 3 → 6	Path: 1 → 2 → 3 → 6	7.5

Kết quả demo

A* tìm path 1 → 2 → 3 → 6 với totalCost = 7.5. Dijkstra trên cùng dataset tìm path 1 → 2 → 3 → 6 với totalCost = 7.5.

5. Độ Phức Tạp

Đại lượng	Phân tích
Worst-case	Có thể tương tự Dijkstra nếu heuristic yếu hoặc không hữu ích.
Thực tế demo	Có thể duyệt ít node hơn khi $h(n)$ định hướng tốt về target.
Bộ nhớ	$O(V + E)$ cho graph, gScore, previous, queue và trace animation.
Tính tối ưu	Giữ được nếu heuristic admissible và graph có trọng số không âm.

6. So Sánh Với Dijkstra

Thuật toán	Path	Total cost	Visited nodes	Ý nghĩa
Dijkstra	$1 \rightarrow 2 \rightarrow 3 \rightarrow 6$	7.5	$1 \rightarrow 2 \rightarrow 0 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow 6$	Baseline chắc chắn.
A*	$1 \rightarrow 2 \rightarrow 3 \rightarrow 6$	7.5	$1 \rightarrow 2 \rightarrow 0 \rightarrow 5 \rightarrow 3 \rightarrow 6$	Ưu tiên node gần target hơn.

7. Checklist Kiểm Thử

- ✓ A* trả cùng totalCost với Dijkstra trên các graph có heuristic admissible.
- ✓ Case $h(n) = 0$ phải cho hành vi như Dijkstra.
- ✓ Heuristic không âm và target có $h(\text{target}) = 0$.
- ✓ Trace relaxedEdges đủ dữ liệu để frontend animate.
- ✓ Dataset demo có nhánh rõ rõ để A* thể hiện lợi thế.